Option 1. Multi-Agent Warehouse Optimization System

The **Multi-Agent Warehouse Optimization System** project aims to design and evaluate an intelligent system where multiple autonomous robots collaborate to efficiently manage warehouse logistics. The primary objective is to develop a simulation environment in which agents learn to perform tasks such as item pickup, transportation, and delivery to packing zones while avoiding collisions and minimizing congestion. Using frameworks like PettingZoo, the system should implement multi-agent reinforcement learning strategy. Unsupervised learning techniques can be incorporated to identify traffic patterns or warehouse bottlenecks.

The result of the project is a working prototype that demonstrates coordinated agent behavior in a dynamic environment. The warehouse environment is grid-based warehouse with robots (agents), shelves (pickup points), drop-off stations (packing zones), obstacles and narrow aisles. Warehouse size, layout (geometry and racks), item distribution, number of robots, and other parameters are defined by the user (random generation is possible for the scenario demonstration). The system must be capable of optimizing key performance metrics such as task throughput, average delivery time, and resource utilization (avoiding collisions, minimizing idle time, distance, etc.).

Define state parameters, reward strategy, use either independent or centralized training approach. You can choose to build custom grid simulator or use environments, such as OpenAI Gym, PettingZoo.

Requirements

1. The system must implement a grid-based warehouse simulation where multiple agents, shelves, obstacles, and delivery points are modeled. The environment should support state updates, agent actions, and episode-based execution.
2. At least one multi-agent reinforcement learning approach must be implemented. The change of optimized performance metrics (task throughput, average delivery time, etc.) during training should be demonstrated in plots. The agents' paths in the beginning and in the end of the training should be demonstrated for the same warehouse configuration.
3. It must be possible to visualize the results of an episode at any time of training (either animation or path visualization) and get the performance metrics of it (number of collisions, idle time, travelled distance, travelled time, etc.).
4. Prepare a 10-minute presentation and demonstration. System demonstration is mandatory, but you can divide 10 minutes according to your needs.

The Report (~10 pages)

Describe system architecture, environment description, used methods, evaluation strategies. The report must include experiments comparing and least two configurations (e.g., different reward functions) with quantitative results and analysis, visual outputs (e.g., agent movement, heatmaps, or trajectories). Comment on the results and provide conclusions. The report must follow the requirements of KTU written works.

Option 2. Robot Arm Control via Reinforcement Learning

The **Robot Arm Control via Reinforcement Learning** project aims to design and evaluate an intelligent system that enables a robotic arm to perform the simple tasks. The primary objective is to develop a simulation environment in which the robotic arm learns to autonomously perform manipulation tasks such as reaching, grasping, and object placement. Using reinforcement learning techniques, the system will learn optimal control policies through interaction with a physics-based simulation environment like PyBullet.

The result of the project is a working prototype that demonstrates the behavior of the robotic arm. The user defines the object to move, the location to place, the obstacles. The agent observes the state of the robot (joint angles, velocities, object positions, etc.) and selects continuous or discrete actions to control joint movements in order to perform user-defined tasks (e.g. user selects an object in the reachable area and defines where the robotic arm must place it). The objects must be placed on the ground carefully (not dropped). There might be multiple objects (obstacles) on the ground (random generation is possible for the scenario demonstration). The system must be capable of optimizing key performance metrics such as task success, the movements (sum of angles, sum of displacement, etc.), task implementation time.

Define state parameters, reward strategy, choose reinforcement learning approach. You can choose to build custom simulator or use environments, such as PyBullet.

Requirements

1. The system must implement a simulation environment where robotic arm movements, objects and obstacles are modeled. The environment should support state updates, agent actions, and episode-based execution.
2. At least one reinforcement learning approach must be implemented. The change of optimized performance metrics (task implementation time, movement performance, etc.) during training should be demonstrated in plots. The motion of the joints of the robotic arm in the beginning and in the end of the training should be demonstrated for the same configuration.
3. It must be possible to visualize the results of an episode at any time of training (either animation or joint parameter visualization) and get the performance metrics of it (episode success, episode time, etc.).
4. Prepare a 10-minute presentation and demonstration. System demonstration is mandatory, but you can divide 10 minutes according to your needs.

The Report (~10 pages)

Describe system architecture, environment description, used methods, evaluation strategies. The report must include experiments comparing and least two configurations (e.g., different reward functions) with quantitative results and analysis, visual outputs (e.g., robotic arm movement, heatmaps, or trajectories). Comment on the results and provide conclusions. The report must follow the requirements of KTU written works.

Option 3. Smart Traffic Light Control with Reinforcement Learning

The **Smart Traffic Light Control with Reinforcement Learning** project aims to design an intelligent traffic management system that dynamically optimizes signal timings at road intersections. Using reinforcement learning, the system learns to adapt traffic light phases based on real-time traffic conditions in a simulated environment such as SUMO.

The result of the project is a working prototype that demonstrates the adaptive behavior under varying traffic patterns, including peak and off-peak scenarios. The agent observes features like vehicle queue lengths, waiting times, and traffic density, and selects actions such as switching or extending signal phases. The objective is to minimize traffic congestion, reduce average waiting time, and improve overall traffic flow efficiency. The user defines the road intersection parameters such as number of lanes, traffic intensity, and others (random generation is possible for the scenario demonstration). The system must be capable of optimizing key performance metrics such as waiting time, queue length, flow efficiency.

Define state parameters, reward strategy, choose reinforcement learning approach. You can choose to build custom simulator or use environments, such as SUMO.

Requirements

1. The system must implement a simulation environment where road intersection and traffic conditions are modelled. The environment should support state updates, agent actions, and episode-based execution.
2. At least one reinforcement learning approach must be implemented. The change of optimized performance metrics (queue length, average waiting time, etc.) during training should be demonstrated in plots. The schedule of signal timings in the beginning and in the end of the training should be demonstrated for the same configuration.
3. It must be possible to visualize the results of an episode at any time of training (either animation or signal schedule visualization) and get the performance metrics of it (max queue length, average queue length, etc.).
4. Prepare a 10-minute presentation and demonstration. System demonstration is mandatory, but you can divide 10 minutes according to your needs.

The Report (~10 pages)

Describe system architecture, environment description, used methods, evaluation strategies. The report must include experiments comparing and least two configurations (e.g., different reward functions) with quantitative results and analysis, visual outputs (e.g., heatmaps or trajectories). Comment on the results and provide conclusions. The report must follow the requirements of KTU written works.

Option 4. Adaptive Inventory Management System

The **Adaptive Inventory Management System** project aims to develop an intelligent decision-making model that optimizes stock control under uncertain and dynamic demand. Using reinforcement learning, the system learns ordering policies that determine when and how much inventory to replenish in order to minimize total costs, including holding, shortage, and ordering costs. The environment simulates real-world supply chain dynamics such as fluctuating customer demand, lead times, and limited storage capacity. Unsupervised learning techniques can be applied to identify patterns in demand data, such as clustering similar demand profiles or detecting seasonality trends

The result of the project is a working prototype that demonstrates the adaptive behavior under different demand and supply patterns. The agent observes the current inventory level, recent demand patterns, and pending orders, and selects actions representing order quantities. The objective is to balance the risk of stockouts against the cost of overstocking. The user defines the holding cost, shortage cost, ordering cost, stock distribution and the demand (random generation based on the distribution with user-defined parameters is possible for the scenario demonstration). The system must be capable of optimizing key performance metrics such as total operational cost, service level (fulfilled demand).

Define state parameters, reward strategy, choose reinforcement learning approach. You can choose to build custom simulator or use environments, such as OpenAI Gym.

Requirements

1. The system must implement a simulation environment where stock supply chain is modelled. The environment should support state updates, agent actions, and episode-based execution.
2. At least one reinforcement learning approach must be implemented. The change of optimized performance metrics (total operational cost, service level, etc.) during training should be demonstrated in plots. The ordering policies in the beginning and in the end of the training should be demonstrated for the same supply-demand configuration.
3. It must be possible to visualize the results of an episode at any time of training (list of orders) and get the performance metrics of it (total operational cost, service level, etc.).
4. Prepare a 10-minute presentation and demonstration. System demonstration is mandatory, but you can divide 10 minutes according to your needs.

The Report (~10 pages)

Describe system architecture, environment description, used methods, evaluation strategies. The report must include experiments comparing and least two configurations (e.g., different reward functions) with quantitative results and analysis, visual outputs (e.g., heatmaps or trajectories). Comment on the results and provide conclusions. The report must follow the requirements of KTU written works.